Course Title: Artificial Intelligence Laboratory

Course Code: CSE 3209

Project Title: Prison Break

Submission Date: 27 April, 2026

Submitted To,	Submitted by,
1. Mehrab Hossain Opi Lecturer, Department of Computer Science and Engineering, Khulna University of Engineering & Technology **2. Waliul Islam Sumon** Lecturer, Department of Computer Science and Engineering, Khulna University of Engineering & Technology	Roll: 2107067 2107078 2107087 Year : **3rd** Term : **2nd** Lab Group : **B1**

1 Objectives

- To develop a complete grid-based prison escape simulation in Godot.
- To implement a Police/Hunter agent controlled by a Fuzzy Logic Inference System.
- To implement a Red prisoner agent controlled by Minimax with Alpha-Beta pruning.
- To implement a Blue prisoner agent controlled by Monte Carlo Tree Search.
- To compare Fuzzy Logic, Minimax, and Monte Carlo Tree Search inside one shared adversarial environment.
- To design a prison map containing walls, boundaries, barriers, CCTV cameras, fire hazards, active exits, dog patrol pressure, and path-blocking objects.
- To implement capture, escape, timeout, elimination, respawn, stamina, ability, danger-map, and score-management logic.
- To visualize AI reasoning through HUD elements, decision overlays, result cards, score panels, replay data, benchmark data, and AI analysis screens.
- To explain the implementation, logic, and design decisions behind every major code component in the project archive.
- To make the project suitable for an AI laboratory demonstration where different agents visibly reason against one another.

2 Introduction

Prison Break is a Godot-based multi-agent artificial intelligence simulation set inside a prison escape scenario. The project shows how different AI techniques behave against each other in the same environment, rather than presenting them as isolated textbook algorithms. The Police/Hunter is the defensive agent and uses a Fuzzy Logic Inference System to decide whether to chase, intercept, investigate, or patrol. The Red prisoner uses Minimax with Alpha-Beta pruning for adversarial escape planning, while the Blue prisoner uses Monte Carlo Tree Search to estimate future escape opportunities through simulations.

The environment is tile-based and includes walls, barriers, doors, CCTV cameras, fire hazards, a dog NPC, active exits, score penalties, and result analysis. These systems create uncertainty and force the agents to make meaningful decisions: the Police/Hunter must interpret alert signals and target priority, while the prisoners must balance escape progress against danger. The report treats the project as an AI decision-visualization system, not only as a game.

Fig 01: Full gameplay screen showing police, prisoners, dog, CCTV, walls, hazards, HUD, and exit.

3 Scope of the Project

- Grid-based prison map and map-generation logic.
- Police/Hunter agent using Fuzzy Logic.
- Red prisoner using Minimax with Alpha-Beta pruning.
- Blue prisoner using Monte Carlo Tree Search.
- Dog NPC with patrol, alert, sniff, chase, latch, and cooldown states.
- CCTV surveillance system with detection, alert contribution, and status effects.
- Walls, boundaries, blocked cells, barriers, fire hazards, active exits, and exit rotation.
- Capture, escape, timeout, elimination, respawn, stamina, ability, and status-effect systems.
- Scoring system with rewards, penalties, assist bonuses, pressure scoring, survival rewards, and performance percentages.
- HUD, overlays, main scene, intro screen, title screen, result screen, pause overlay, and AI analysis screen.
- Replay export/import and benchmark output tools.
- Project specification notes, UI prototype, imported assets, and Godot engine metadata.

4 Tools and Technologies Used

Table 1: Tools and technologies used

Component	Technology
Game Engine	Godot
Programming Language	GDScript
AI Technique 1	Fuzzy Logic Inference System
AI Technique 2	Minimax with Alpha-Beta Pruning
AI Technique 3	Monte Carlo Tree Search
Environment Model	Grid-based simulation
UI	HUD, overlays, result screen, AI analysis page
Architecture	Signal/event-driven simulation

Table 2: Important files and folders

File or Folder	Purpose
project.godot	Godot configuration, autoloads, main scene, viewport settings, input actions, and project identity.
ai/	AI controllers and configuration resources for Fuzzy Logic, Minimax, and MCTS.
autoloads/	Global services including EventBus, random seed service, sound manager, sprite loader, and user settings.
core/	Simulation loop, scoring, grid/cost/danger maps, AI decision recorder, replay, and benchmark helpers.
data/	Configuration resources for AI, simulation, scoring, and agent statistics.
gameplay/	Agents, dog NPC, abilities, status effects, CCTV, doors, fire, and intro video support.
scenes/	Main game, intro, title, result screen, and Godot scene files.
ui/	HUD, panels, overlays, pause overlay, transient effects, and debug displays.
world/	Map generation, exit rotation, grid rendering, path overlay, danger heatmap, and vision overlay.
tools/	Benchmark runner, replay exporter/importer, and step debugger.
assets/	Visual, audio, icon, portrait, and video assets.

prison-break-ui.html	HTML UI prototype showing the intended interface style.

5 System Architecture

The project uses a centralized simulation architecture. The game scene constructs the grid renderer, overlays, HUD, tick clock, simulation loop, benchmark runner, replay exporter/importer, and debugging tools. The simulation loop coordinates perception, CCTV, dog behaviour, hazards, danger/cost maps, AI actions, movement, capture/escape checks, scoring, replay snapshots, and final results.

A signal/event-driven design is used through the EventBus autoload. Capture, escape, camera detection, dog engagement, tick updates, danger-map updates, and finalization events are emitted once and consumed by the HUD, scoring system, replay tools, and AI decision recorder.

Game Scene
|
+— Tick Clock
+— Simulation Loop
|
+— Grid Engine / Cost Map / Danger Map
+— EventBus
+— Scoring System
+— AI Decision Recorder
|
+— Police/Hunter: Fuzzy Logic Controller
+— Red Prisoner: Minimax Controller
+— Blue Prisoner: MCTS Controller
+— Dog NPC and CCTV System
|
+— HUD, Overlays, Result Screen, Replay, Benchmark

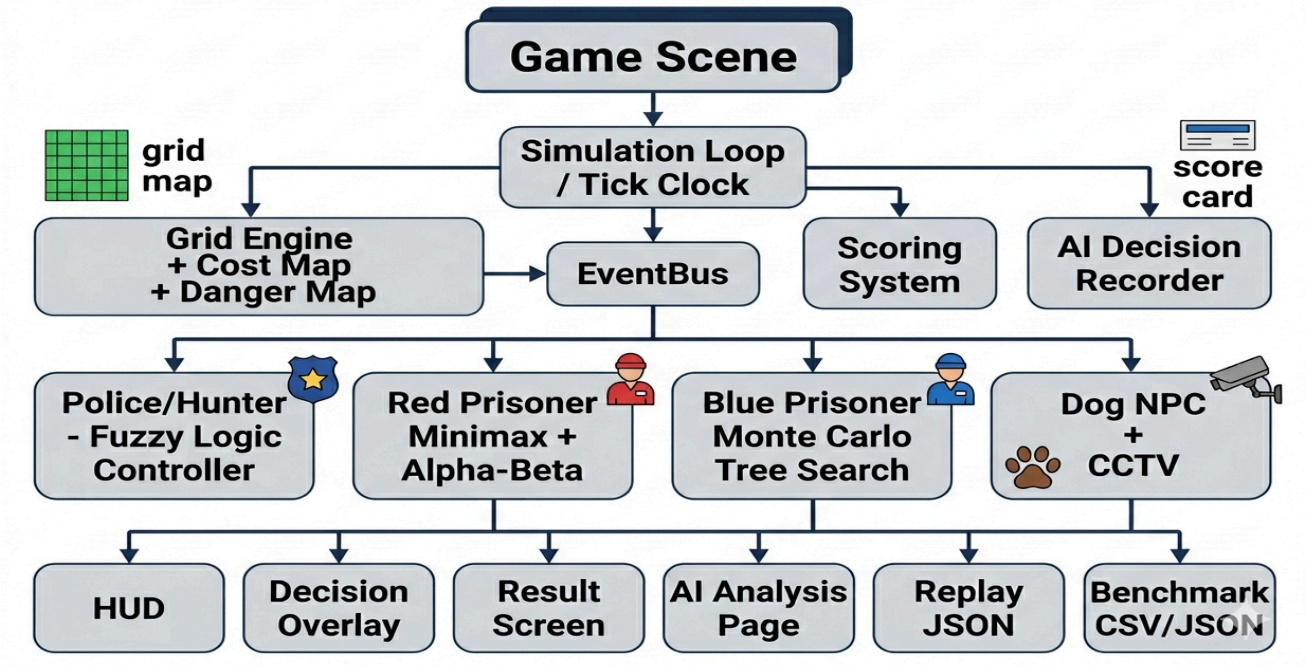


Fig 02: Overall system architecture.

6 Game Environment Design

6.1 Grid-Based Map

The environment is a tile grid. Agents store grid positions and move by selecting neighboring walkable cells. Using a discrete grid makes Minimax, MCTS, fuzzy chasing, CCTV visibility, dog danger, and pathfinding operate on the same coordinate model.

6.2 Walls, Boundaries, Doors and Barriers

Walls and boundaries block movement and force route planning. Doors and barriers restrict or delay paths, making the escape problem more strategic without making the map unfair or unreachable.

6.3 Fire Hazards

Fire hazards seed the danger map, create penalties, and can trigger burning or elimination pressure. They force prisoners to trade direct progress against safety, while indirectly helping the Police/Hunter by narrowing safe routes.

6.4 CCTV Cameras

CCTV cameras monitor selected zones using range and visibility checks. Detection can apply a detected status effect, create score penalties, and increase Police/Hunter alert, making CCTV an information source for fuzzy decision-making.

6.5 Active Exit

Prisoners must reach the active exit. Exit rotation and decoy exits make the objective dynamic. The Police/Hunter uses exit threat when deciding whether to chase directly or intercept the escape route.

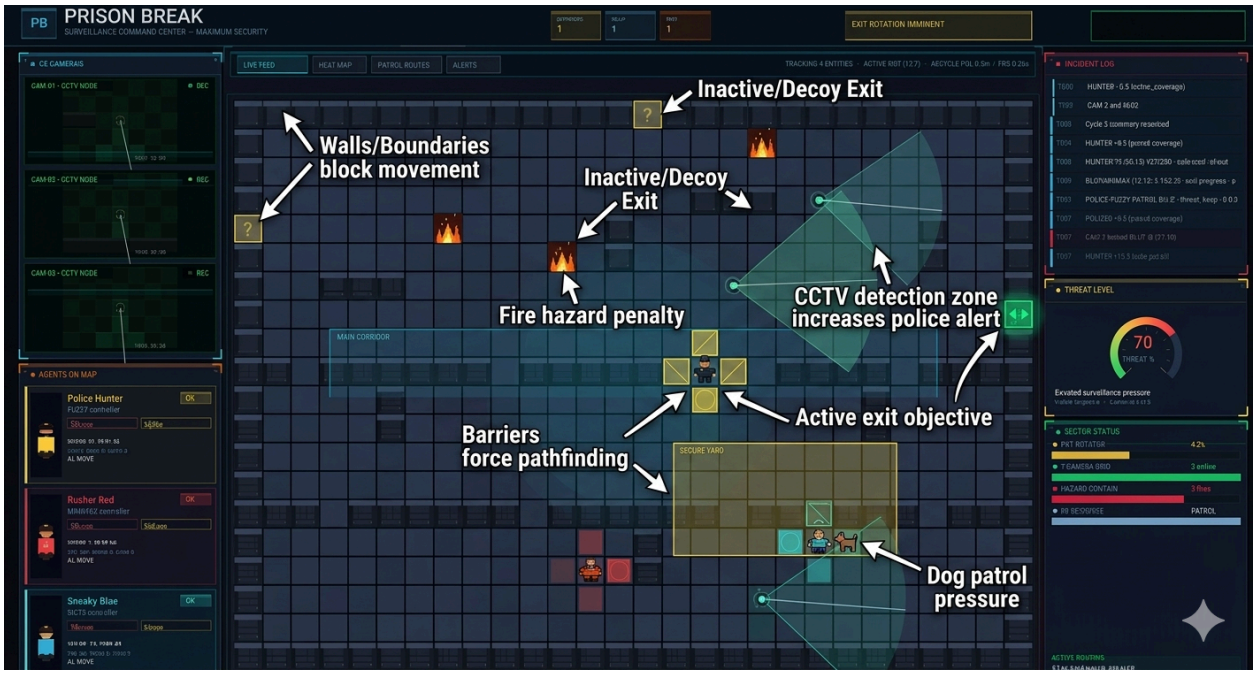


Fig 03: Annotated prison map showing walls, boundaries, fire hazards, CCTV detection zones, barriers, active exit, inactive/decoy exits, dog patrol pressure, and safe-cost route planning.

7 Agent Design and AI Behaviour

7.1 Police/Hunter Agent - Fuzzy Logic AI

The Police/Hunter is the main defensive AI. It uses fuzzy reasoning because its decisions depend on uncertain signals such as distance, alert level, CCTV hints, dog signals, exit threat, stealth, target priority, and capture opportunity.

Table 3: Police/Hunter fuzzy behaviours

Behaviour	Meaning
Chase	Move toward the selected prisoner or a capture approach tile.
Intercept	Move toward an exit-side or predicted escape tile.
Investigate	Move toward suspicious signals, last known position, or CCTV hint.

Patrol	Guard or move to patrol targets when threat is low.

Fig 04: Police chasing or intercepting prisoner.

7.2 Red Prisoner - Minimax Agent

The Red prisoner is a rusher-style escape agent controlled by Minimax with Alpha-Beta pruning. It treats the Police/Hunter as an adversarial opponent and chooses the move that maximizes escape value while minimizing risk.

Table 4: Red prisoner design

Feature	Description
Agent	Red prisoner / RusherRed
Algorithm	Minimax with Alpha-Beta pruning
Goal	Escape through the active exit
Strength	Strategic planning against an opponent model
Weakness	Depends on search depth and heuristic quality

7.3 Blue Prisoner - Monte Carlo Tree Search Agent

The Blue prisoner is a stealth-oriented escape agent controlled by MCTS. It estimates future outcomes through rollouts and chooses moves using visit counts, reward values, and exploration/exploitation balance.

Table 5: Blue prisoner design

Feature	Description
Agent	Blue prisoner / SneakyBlue
Algorithm	Monte Carlo Tree Search
Goal	Escape while avoiding Police/Hunter, dog, CCTV, and hazards
Strength	Simulation-based planning under uncertainty
Weakness	Depends on rollout count and reward design

7.4 Dog NPC

The dog is a mobile environmental threat. It patrols waypoints, detects suspicious prisoners, sniffs, chases, latches, and then returns through a release/cooldown cycle. It increases police alert and changes the danger map.

Table 6: Dog states

State	Meaning
Idle	Temporary inactive or transition state.
Patrol	Follows patrol waypoints.
Alert	Detected suspicious signal.
Sniff	Investigates nearby area.
Chase	Moves toward a prisoner.
Latched	Catches or slows a prisoner.
Release Cooldown	Resets after latch/release.

8 Main Simulation Loop

The simulation loop is implemented in core/simulation_loop.gd. It uses a 60-second match duration and 0.25-second ticks, producing deterministic updates that can be replayed, scored, and analyzed.

Start Tick

→ update perception, CCTV visibility, camera alert floor, and alert decay

→ rebuild danger and cost maps

→ ask Police/Hunter Fuzzy AI for decision

→ ask Red Minimax AI for decision

→ ask Blue MCTS AI for decision

→ resolve movement, sprint, wait, interact, or ability actions

→ update dog, fire, doors, status effects, and cooldowns

→ rebuild post-move danger and cost maps

→ apply periodic scoring tick

→ check escape, decoy exit penalty, fire elimination, capture, timeout, and final outcome

→ event-based scoring reacts to escape/capture/CCTV/dog/fire signals

→ record metrics, AI analysis, and replay snapshot

→ update HUD and overlays

Next Tick

9 Pathfinding and Movement Logic

Movement is tile-based. Agents generate legal neighboring tiles, filter blocked or occupied positions, evaluate risk from the danger/cost map, and then move toward goals such as prisoners, exits, patrol targets, or investigation tiles. The shared AIController utilities provide A* baseline paths, oscillation avoidance, fallback movement, and position lookup.

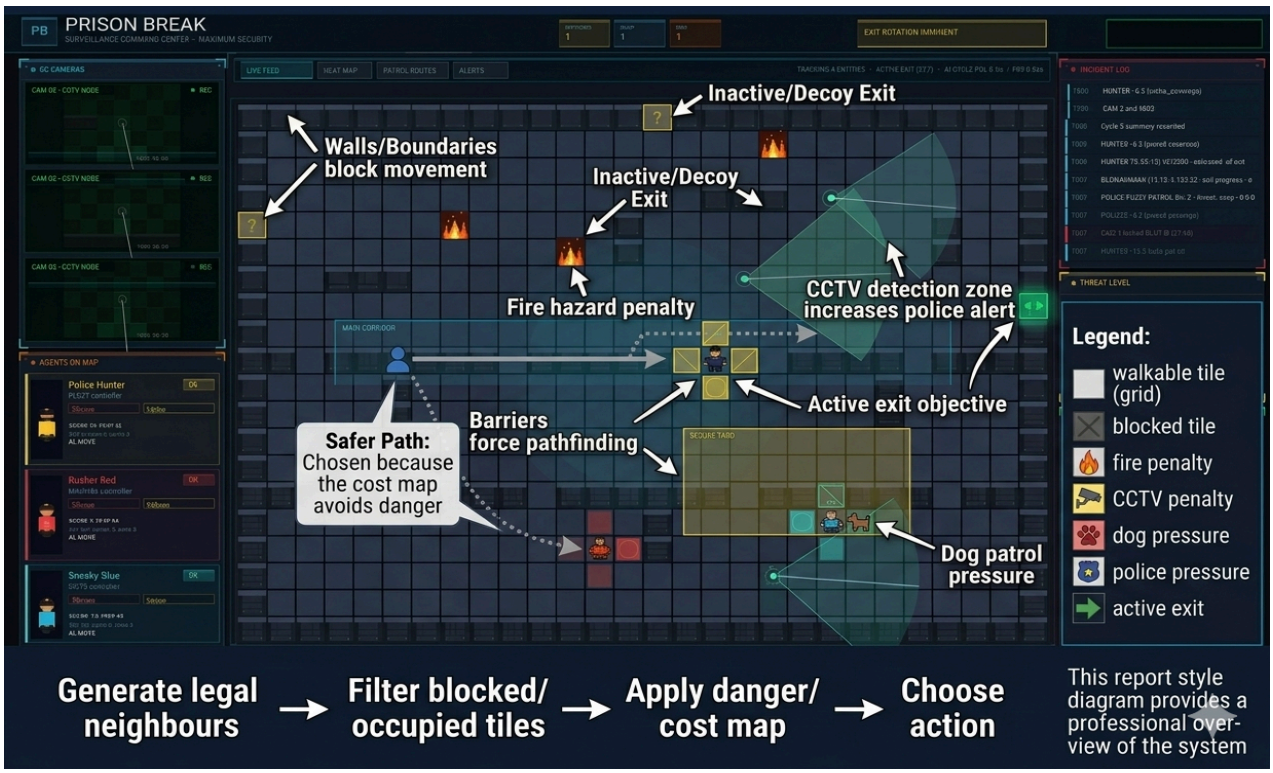


Fig 5: Grid movement and pathfinding example.

10 Fuzzy Logic System for Police/Hunter

10.1 Why Fuzzy Logic Is Used

The Police/Hunter receives partial, uncertain, and changing information during the match. A strict if-else system would make the police behaviour too sudden, for example instantly changing from patrol to chase at one fixed distance. Fuzzy logic is used because it allows the police to reason with degrees of truth, such as "the prisoner is partly near", "the alert level is moderately high", or "the CCTV confidence is strong".

In this project, fuzzy logic is directly connected to the Police/Hunter agent through gameplay/police_hunter.gd, where the police is assigned FuzzyController as its AI controller. Every simulation tick, core/simulation_loop.gd calls choose_action() on the police controller. The controller observes prisoners, reads CCTV hints, evaluates exit threat, selects the most threatening prisoner, calculates fuzzy behaviour scores, chooses a final behaviour, and produces a movement action.

The full fuzzy decision pipeline is:

PoliceHunter

→ FuzzyController.choose_action()

→ collect active prisoners

→ update vision/noise observations

→ update CCTV hint

→ calculate target priority

→ select Red or Blue as target

→ fuzzify distance, alert level, exit threat, and CCTV confidence

→ calculate Chase, Intercept, Investigate, and Patrol scores

→ choose the highest-scoring behaviour

→ convert behaviour into a strategic movement tile

→ move using A* pathfinding

→ capture is checked by SimulationLoop when police becomes adjacent.

10.2 Fuzzy Inputs

Table 7: Police/Hunter fuzzy inputs and supporting priority signals

Input	Used In	Meaning
Police-prisoner distance	Fuzzy behaviour scoring	Converted into near, medium, and far membership values.
Police alert level	Fuzzy behaviour scoring	Converted into suspicious and alarmed membership values.
Exit threat / escape urgency	Fuzzy behaviour scoring	Converted into low, medium, and high exit-threat membership values.
CCTV confidence	Fuzzy behaviour scoring	Converted into weak, medium, and strong CCTV confidence membership values.
Visibility confidence	Target priority	Increases when police has line-of-sight to prisoner.
Prisoner stealth	Target priority	Low stealth increases police priority toward that prisoner.
Capture opportunity	Target priority	High when police is already close enough to capture soon.
Target lock	Target selection stability	Prevents police from switching target every tick.
Dog influence	Indirect input	Dog spotting increases police alert; latch slows prisoner.

10.3 Membership Functions

The fuzzy controller uses membership functions to convert crisp numeric values into fuzzy degrees between 0.0 and 1.0.

Distance is interpreted as:

- Near

- Medium

- Far

Alert level is interpreted as:

- Suspicious

- Alarmed

Exit threat is interpreted as:

- Low

- Medium

- High

CCTV confidence is interpreted as:

- Weak

- Medium

- Strong

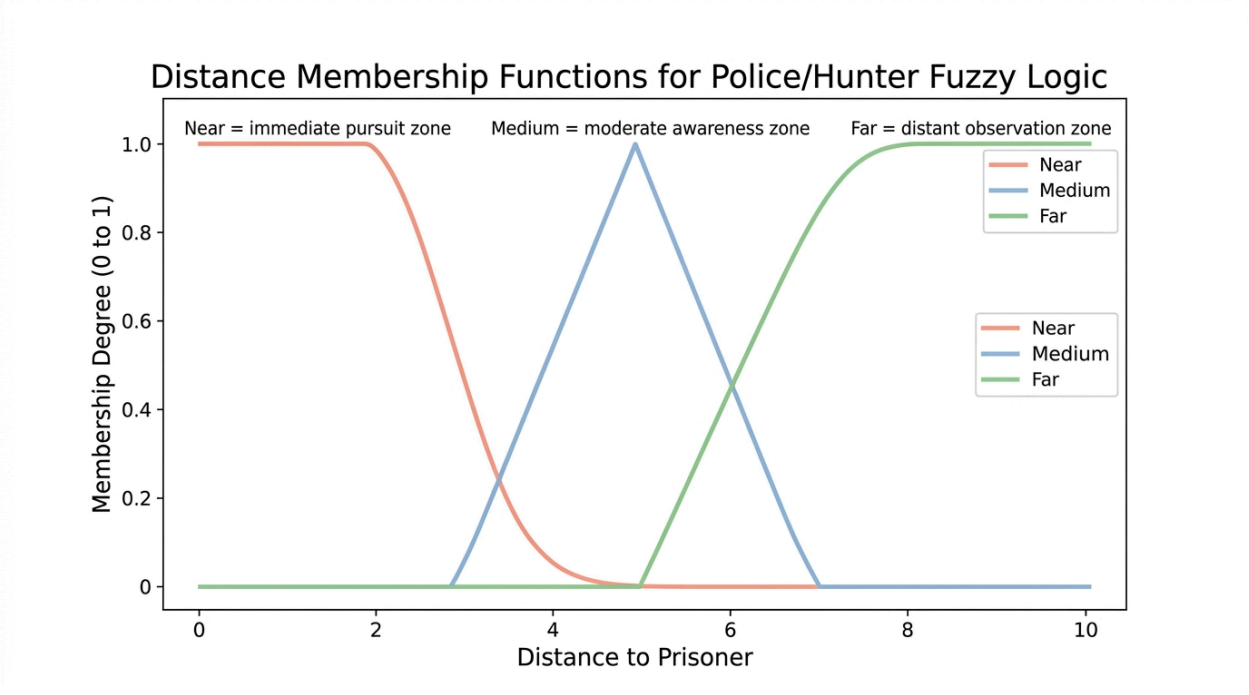


Fig 6: Distance membership functions.

The distance membership values are loaded from data/ai/fuzzy_config.tres. The implemented distance thresholds are:

Near:

- Full near membership up to distance 3.0.

- Near membership becomes zero at distance 5.5.

Medium:

- Starts from distance 3.5.

- Peaks at distance 7.0.

- Becomes zero at distance 11.0.

Far:

- Starts from distance 7.0.

- Becomes fully far at distance 12.0.

This overlapping design allows smooth transition. For example, a prisoner at distance 4 may be partly near and partly medium instead of suddenly changing from chase to investigate.

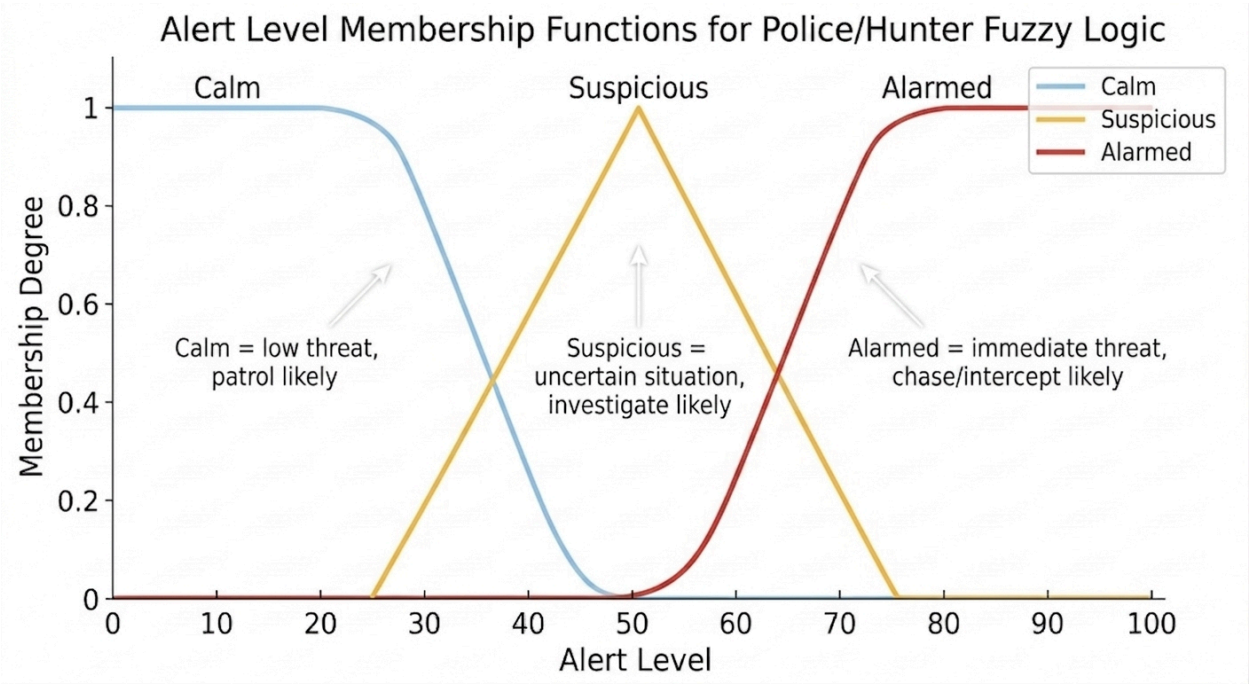


Fig 7: Alert membership functions.

The alert level is internally stored as a normalized value from 0.0 to 1.0. The UI may display it as a percentage, but the fuzzy controller uses normalized values.

Suspicious alert:

- Starts around 0.20.

- Peaks at 0.35.

- Fades by 0.75.

Alarmed alert:

- Starts at 0.50.

- Becomes fully active at 0.85.

Low alert mainly supports patrol through the expression (1.0 - alert_level).

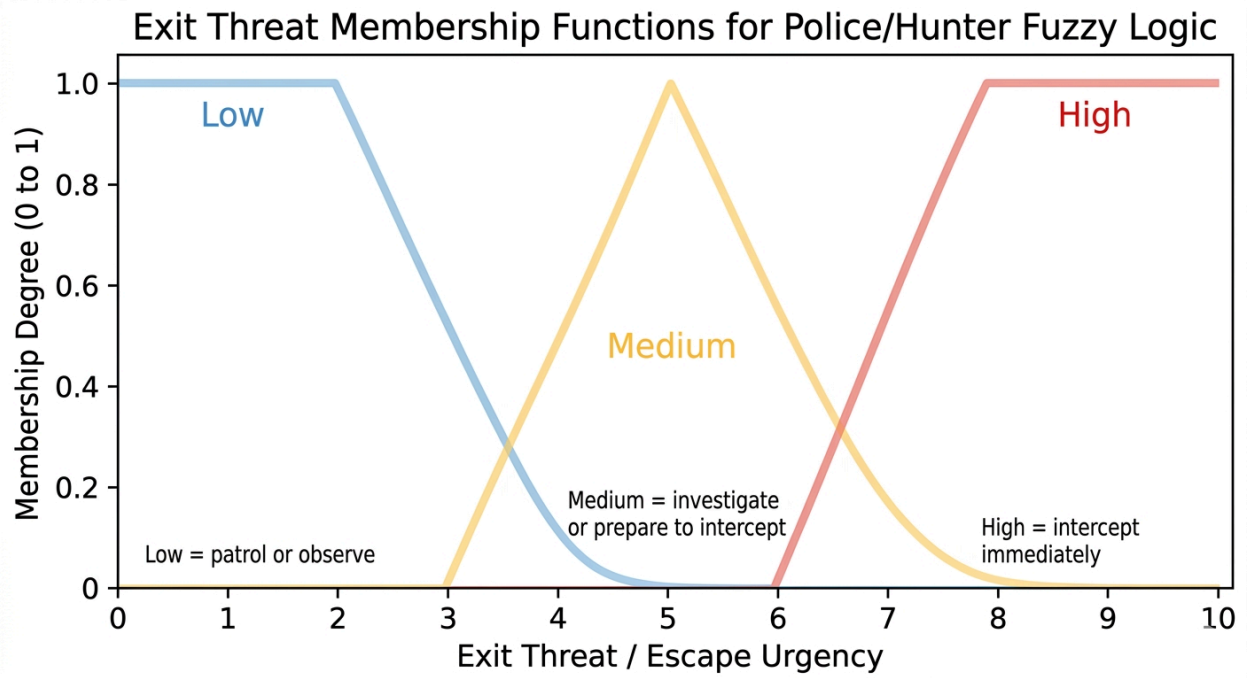


Fig 8: Implemented exit threat membership functions for Police/Hunter fuzzy logic..

Exit threat is now implemented as a direct fuzzy input. It represents how dangerous the selected prisoner is based on distance from the active exit. A prisoner close to the active exit creates high exit threat because escape may happen soon.

Exit threat is normalized into an escape urgency value and then fuzzified into:

Low exit threat:

- The prisoner is far from the active exit.

- Police can patrol or observe.

Medium exit threat:

- The prisoner is approaching the active exit.

- Police may investigate or prepare to intercept.

High exit threat:

- The prisoner is very close to the active exit.

- Police should intercept immediately.

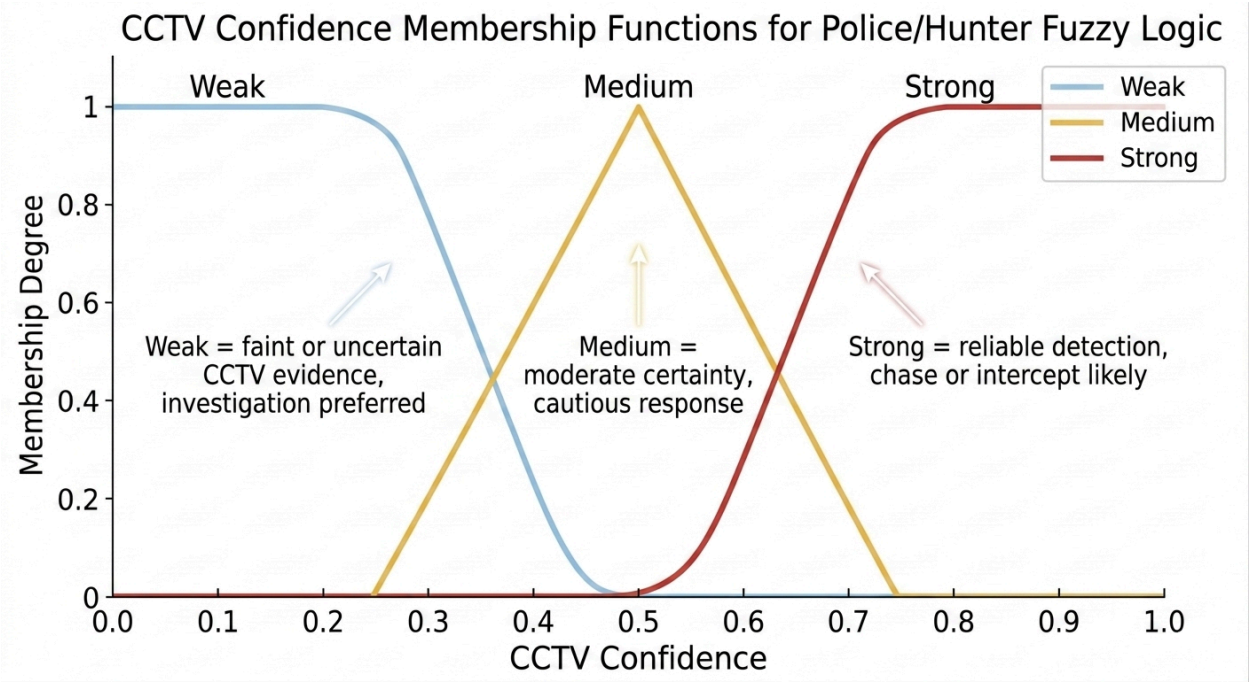


Fig 9: Implemented CCTV confidence membership functions for Police/Hunter fuzzy logic.

CCTV confidence is now also implemented as a direct fuzzy input. When CCTV detects a prisoner, the police receives a CCTV hint with confidence. This confidence is fuzzified into:

Weak CCTV confidence:

- The camera signal is old, uncertain, or weak.

- Police should not fully commit to chase.

Medium CCTV confidence:

- The signal is moderately reliable.

- Police may investigate the hinted area.

Strong CCTV confidence:

- The camera recently detected the prisoner.

- Police can chase or intercept more confidently.

10.4 Rule Base and Behaviour Scores

Before choosing chase, intercept, investigate, or patrol, the fuzzy controller first selects which prisoner should be targeted. This is necessary because the game has two prisoners: Red and Blue. The police should not randomly switch between them every tick.

For each active prisoner, the controller calculates a target-priority score using:

target_priority =

capture_opportunity

+ exit_threat

+ visibility_confidence × 2.0

+ cctv_confidence × 1.7

+ low_stealth_bonus

- police_distance_cost

- target_switch_penalty

The components mean:

capture_opportunity:

High when the police is already close enough to capture soon.

exit_threat:

High when the prisoner is close to the active exit.

visibility_confidence:

High when police has direct line-of-sight to the prisoner.

noise_confidence:

Used when the prisoner is noisy enough to be detected.

cctv_confidence:

High when CCTV recently detected the prisoner.

low_stealth_bonus:

Low-stealth prisoners are easier to detect and chase, so they receive higher priority.

police_distance_cost:

Farther prisoners reduce priority because they are harder to catch.

target_switch_penalty:

Prevents the police from switching targets every tick unless another prisoner becomes much more urgent.

10.5 Rule Base and Behaviour Scores

Table 8: Updated Fuzzy Rule Base

Rule / Score Component	Behaviour Effect
near_mu × w_chase	Increases Chase when prisoner is close.
medium_mu × alarmed_mu	Increases Chase when target is medium distance and police is alarmed.
alert_level × alert_chase_weight	Smoothly increases Chase as alert level rises.
alert_chase_base_bias × alert_level	Adds extra chase bias during alert situations.
exit_high_mu	Strongly increases Intercept when prisoner is close to active exit.
exit_medium_mu	Supports Investigate or prepared Intercept.
exit_low_mu	Supports Patrol or observation.
cctv_strong_mu	Increases Chase or Intercept because camera information is reliable.
cctv_medium_mu	Increases Investigate because the signal is useful but not fully certain.
cctv_weak_mu	Supports cautious behaviour instead of full chase.
medium_mu × max(suspicious_mu, alarmed_mu × 0.60)	Increases Investigate for uncertain medium-distance targets.
far_mu × suspicious_mu × 0.65	Increases Investigate for far but suspicious targets.
far_mu × max(alarmed_mu, exit_high_mu)	Increases Intercept for far but urgent targets.
far_mu × (1.0 - alert_level)	Supports Patrol when target is far and alert is low.
distance <= 2	Adds Chase bonus because capture is close.
recent last-seen tick	Adds Investigate bonus.

The inspected controller emits both HUD-friendly fuzzy debug data and result-screen fuzzy decision data. It includes target scores for Red and Blue and behaviour scores for chase,

intercept, investigate, and patrol. This supports an AI analysis page that can draw a fuzzy inference tree instead of only showing a final movement tile.

10.6 Defuzzification and Final Behaviour Selection

After calculating the four behaviour scores, the fuzzy controller performs winner-takes-all defuzzification. This means the behaviour with the highest score becomes the final crisp police behaviour.

The possible final behaviours are:

- Chase

- Intercept

- Investigate

- Patrol

For example, if the scores are:

Chase = 2.03

Intercept = 1.45

Investigate = 0.25

Patrol = 0.10

then the selected behaviour is Chase.

This project does not use centroid defuzzification because the output is not a continuous numeric value such as speed or angle. The output is a discrete tactical behaviour. Therefore, score comparison is more suitable for this implementation.

10.7 Police Fuzzy Decision Tree

The Police/Hunter decision tree is not a Minimax tree. It is a fuzzy inference structure that explains how target selection and behaviour scores produce a final action.

Police Fuzzy Root
|
+— Input Collection
| +— police position
| +— Red position
| +— Blue position
| +— active exit
| +— police alert level
| +— CCTV hint

| +— last seen positions
|
+— Target Priority Calculation
| +— capture opportunity
| +— exit threat
| +— visibility/noise confidence
| +— CCTV confidence
| +— low-stealth bonus
| +— distance cost
| +— target switch penalty
|
+— Target Selection
| +— Red target score
| +— Blue target score
| +— selected prisoner
|
+— Fuzzification
| +— distance → near / medium / far
| +— alert → suspicious / alarmed
| +— exit threat → low / medium / high
| +— CCTV confidence → weak / medium / strong
|
+— Behaviour Score Calculation
| +— chase score
| +— intercept score
| +— investigate score
| +— patrol score
|
+— Defuzzification
| +— highest score wins
|
+— Action Generation
+— strategic tile
+— A* next step
+— move / sprint / wait

10.8 Converting Fuzzy Behaviour into Movement

After the behaviour is selected, the controller converts it into a strategic target tile:

Table 9: Fuzzy movement

Behaviour	Movement Target
Chase Intercept	Prisoner's current position or nearby capture approach tile. A tile ahead on the prisoner's path toward the active exit.
Investigate	Last seen position or CCTV hinted position.
Patrol	Strategic guard tile near exit or likely escape route.

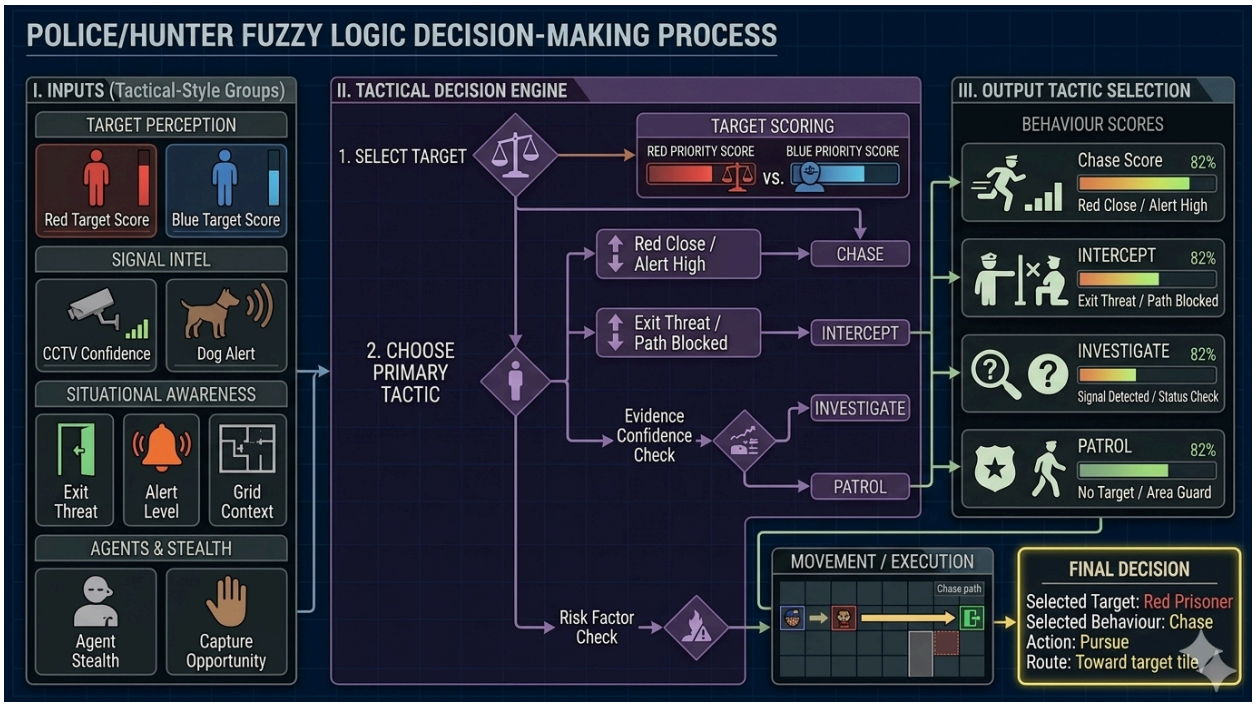


Fig 10: Police Fuzzy Logic Decision-Making Process

11 Minimax Algorithm for Red Prisoner

11.1 Purpose and Algorithm Overview

The Red prisoner uses Minimax with Alpha-Beta pruning because it is designed as a rusher-style agent that plans against police response. Red is the maximizing player because it wants to increase escape value, while the Police/Hunter is modeled as the

minimizing opponent because it tries to reduce Red's escape value. Alpha-Beta pruning removes branches that cannot affect the final selected move.

11.2 State Representation

The Minimax controller considers Red position, Police/Hunter position, Blue position, active exit, danger map, fire tiles, wall penalties, stamina, capture distance, and current search depth. Legal moves are generated from free neighboring grid cells.

11.3 Evaluation Function

The evaluation combines objective progress and risk: Score = exit-progress reward - danger penalty - guard pressure - fire penalty - wall penalty + stamina value. Alpha-Beta pruning removes branches that cannot improve the final decision, allowing deeper planning within real-time limits.

The implemented evaluation function combines escape progress, danger, police pressure, stamina, fire risk, and wall penalty.

score =

w_exit × (-distance_to_exit)

+ w_risk × (-danger_cost)

+ w_opp × (-1 / max(distance_to_police, 1))

+ w_stam × stamina_ratio

- guard_penalty

- fire_penalty

- wall_penalty

This means Red prefers moves that reduce distance to the active exit, avoid danger, maintain stamina, and stay away from police pressure.

11.4 Alpha-Beta Pruning

Alpha-Beta pruning avoids evaluating branches that cannot change the final decision. Alpha stores the best value found for the maximizer. Beta stores the best value found for the minimizer. If beta becomes less than or equal to alpha, the remaining branch can be pruned.

```
Red root
 +— Red move A
 |   +— Police response A1 -> score
 |   +— Police response A2 -> score
 +— Red move B
```

| +— Police response B1 -> score
| +— Police response B2 -> pruned
+— Red move C
   +— Police response C1 -> score

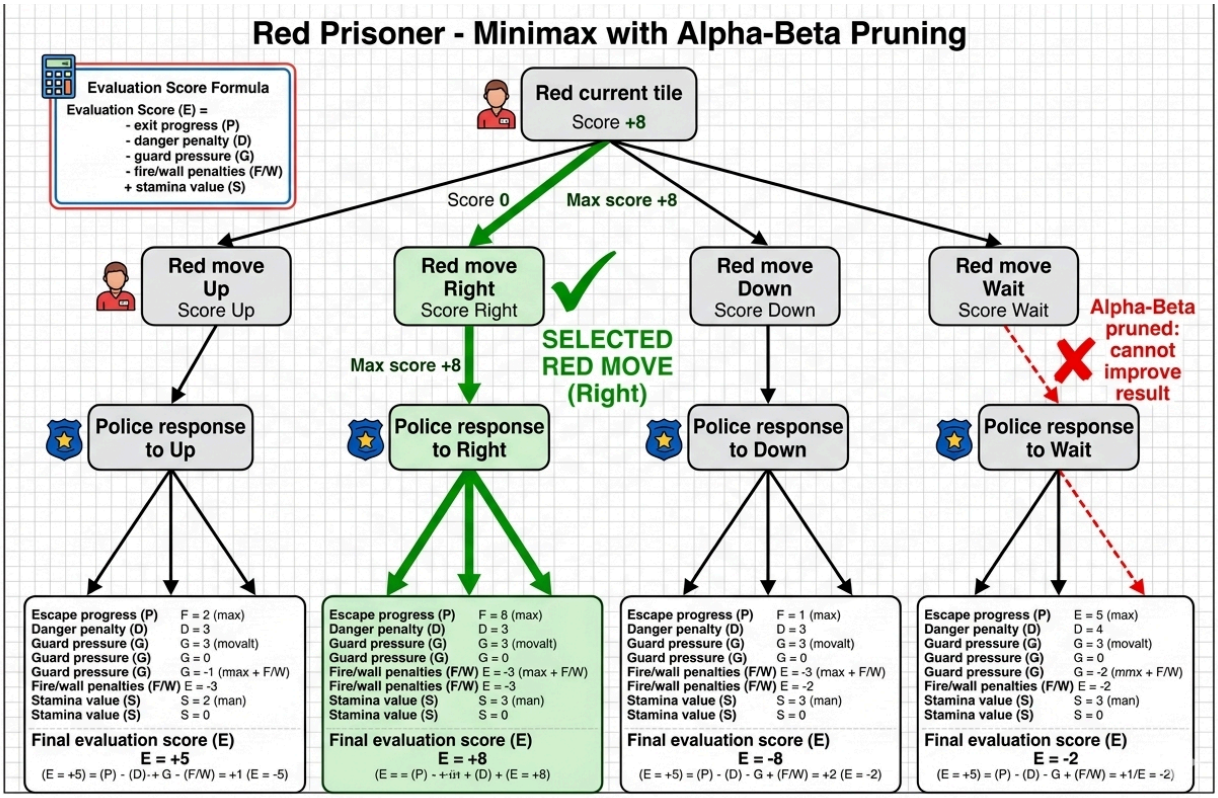


Fig 11: Red prisoner Minimax decision tree from AI analysis page.

12 Monte Carlo Tree Search for Blue Prisoner

12.1 Purpose

Blue uses MCTS because exact future prediction is expensive. It evaluates moves through Selection, Expansion, Simulation/Rollout, and Backpropagation. Each candidate move stores visits, average reward, and UCT-style value.

12.2 Four Phases of MCTS

Selection -> Expansion -> Simulation / Rollout -> Backpropagation

12.3 UCT Formula

UCT = average_reward + C * sqrt(ln(parent_visits) / child_visits).

The first part, average_reward, represents exploitation because it prefers moves that performed well before. The second part represents exploration because it gives

less-visited moves a chance. The exploration constant C controls the balance between trying new moves and reusing good moves.

Rollout scoring considers escape progress, distance to exit, police safety, dog/CCTV/fire danger, stamina cost, active exit rotation, and survival. Simulated police movement makes Blue's planning less over-optimistic.

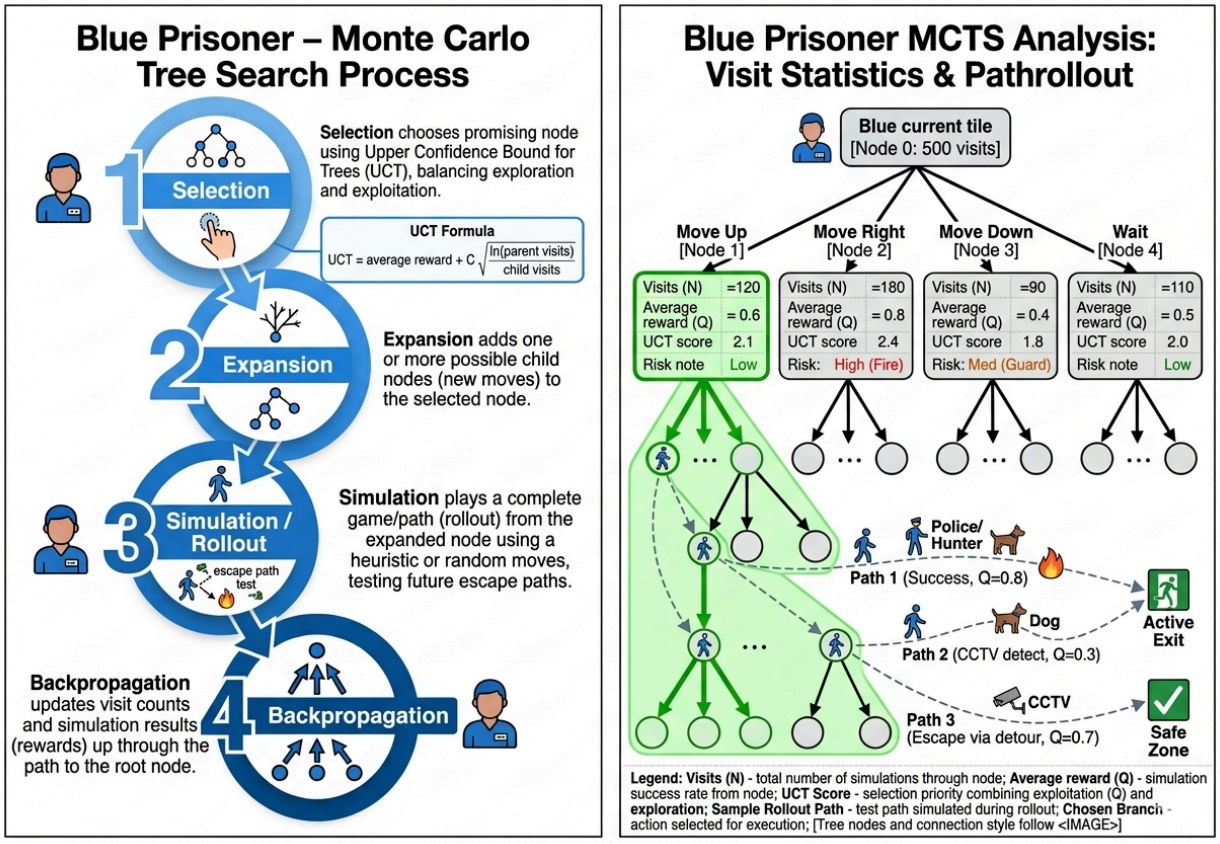


Fig 12: Blue prisoner MCTS tree/statistics from AI analysis page.

13 CCTV Surveillance Logic

CCTV checks whether a prisoner is within camera range, inside field-of-view, and visible through raycast. If detection succeeds, the prisoner receives detected status, score penalty, stamina/stealth pressure, and a camera detection event is emitted. The scoring system then gives police a CCTV assist and provides CCTV hint information to the fuzzy controller.

Prisoner enters CCTV range
-> camera visibility check
-> detection confidence / event
-> prisoner detected status and penalty

-> Police/Hunter alert increases
-> fuzzy target priority and behaviour may change

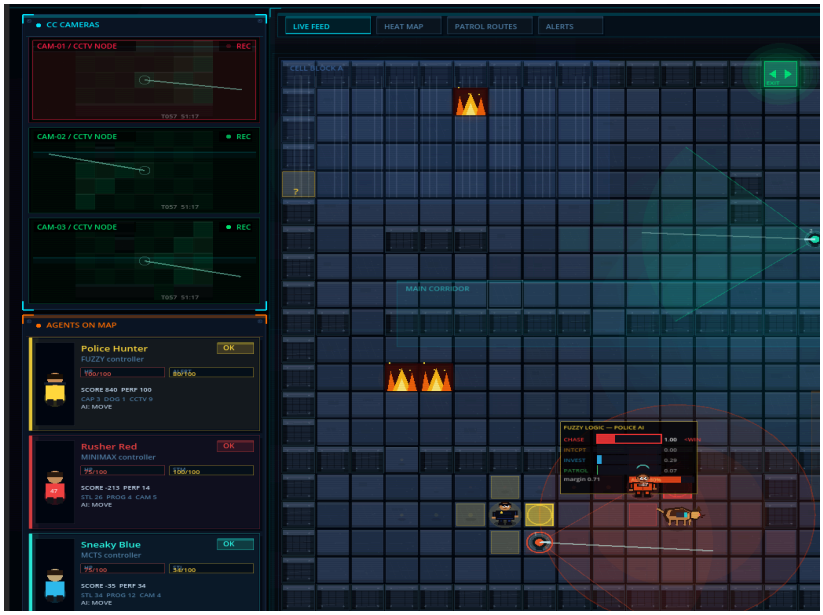


Fig 13: CCTV detecting a prisoner.

14 Dog Detection and Chase Logic

The dog uses a state machine with Idle, Patrol, Alert, Sniff, Chase, Latched, and Release Cooldown states. It detects prisoners using noise/proximity and line-of-sight. When it spots a prisoner, police alert increases. If it reaches Manhattan distance <= 1, it latches onto the prisoner, applies a pinned/slow effect, and creates dog-assist scoring.

Patrol -> detects noise or proximity -> Alert -> Sniff -> Chase -> Latched -> Release Cooldown -> Patrol

Fig 14: Dog detecting or chasing prisoner.

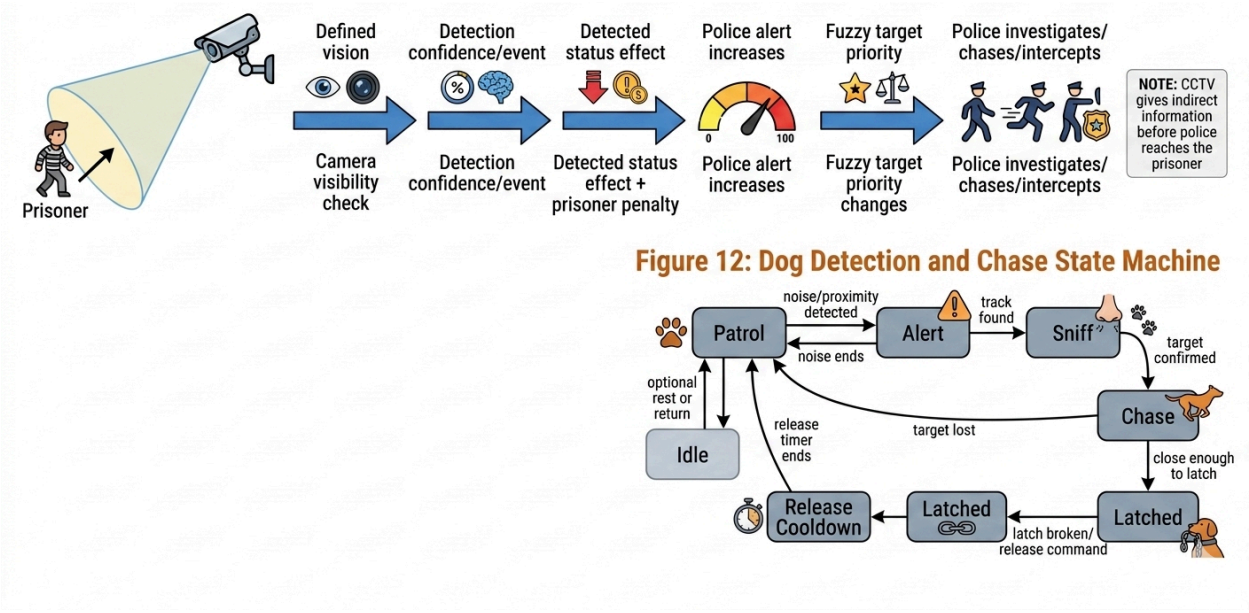


Fig 15: CCTV Surveillance and Police Alert Flow

15 Walls, Doors, Barriers, Fire, and Environmental Hazards

Table 10: Environmental constraints and AI effect

Element	Gameplay Effect	AI Effect
Wall / Boundary	Blocks movement and shapes corridors.	Forces route planning.
Door / Barrier	Restricts or delays paths.	Changes available actions and path cost.
Fire	Creates danger, penalties, or elimination pressure.	Discourages risky direct paths.
CCTV	Detects prisoners.	Increases police information and alert.
Dog	Mobile threat.	Changes prisoner safety and police alert.
Active Exit	Escape objective.	Drives prisoner goals and police interception.
Decoy/Inactive Exit	Non-goal exit.	Can mislead path choices or add route penalties.

16 Capture, Escape, and Timeout Logic

16.1 Capture Logic

The actual capture rule is implemented in core/simulation_loop.gd, not directly inside the fuzzy controller. The fuzzy controller decides the police target, behaviour, and movement tile. After movement is resolved, SimulationLoop checks the distance between each active police agent and each active prisoner.

A prisoner is captured when:

Manhattan distance between police and prisoner <= 1

and prisoner.capture_cooldown_ticks == 0

This means passive simulation capture uses cardinal adjacency: left, right, up, or down. After capture, prisoner.capture_count and police.capture_count are updated, the agent_captured event is emitted, and the prisoner either respawns or is eliminated. A prisoner is eliminated after three captures. If not eliminated, the prisoner respawns and receives temporary capture cooldown so that the police cannot instantly capture the same prisoner again.

16.2 Escape Logic

A prisoner escapes by reaching the active exit. Escaped prisoners become inactive and receive escape scoring, while the Police/Hunter receives an escape-allowed penalty.

16.3 Timeout Logic

If the 60-second timer expires, remaining prisoners are timed out. Timeout is treated separately from physical capture for result accuracy.

Table 11: Outcome categories

Outcome	Meaning
Prisoners Win	Both prisoners escape.
Police Wins	Prisoners are captured, eliminated, or fail to escape.
Partial Escape	At least one prisoner escapes but not all prisoners escape.
Timeout	Match duration ends before full escape.

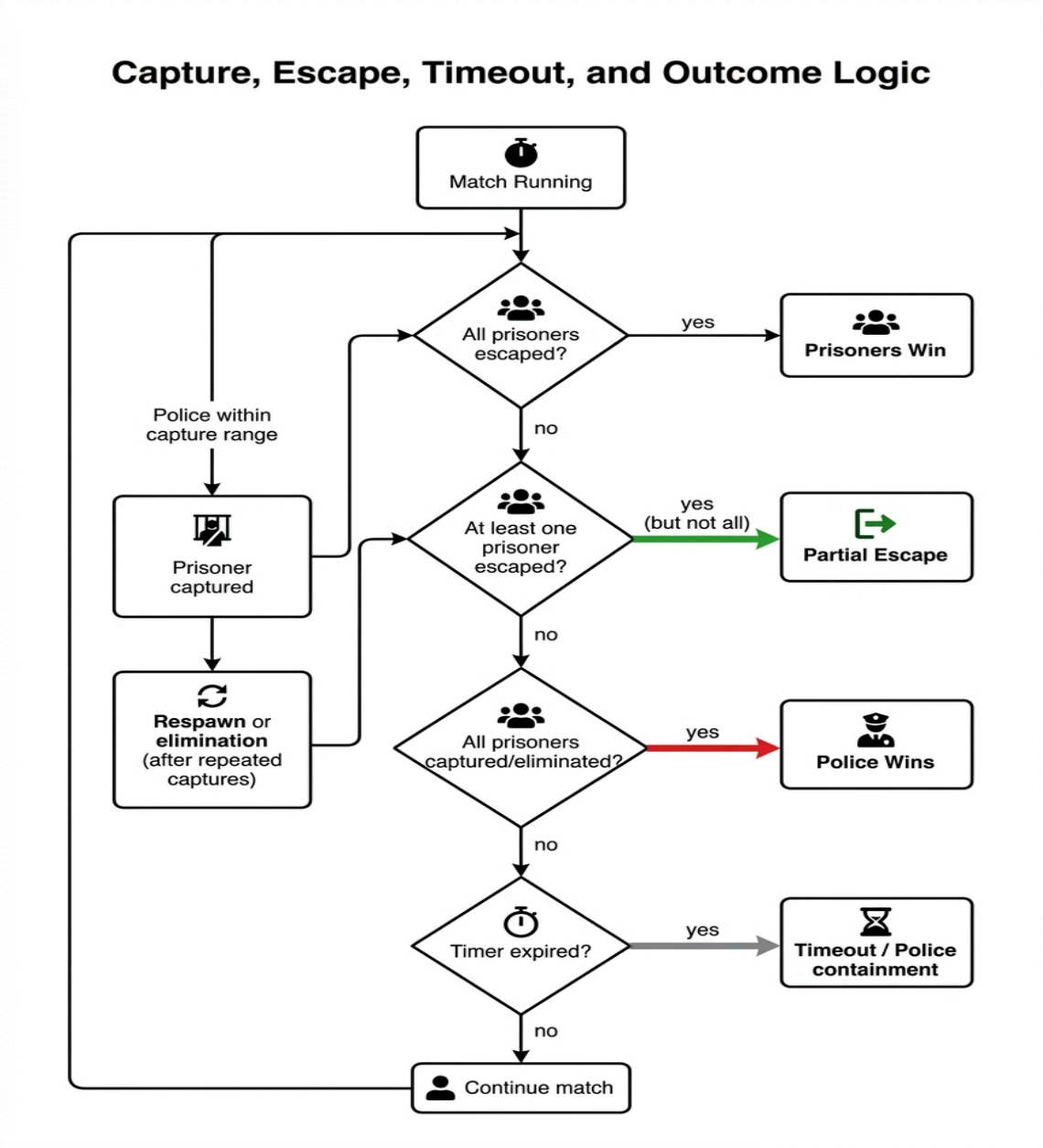


Fig 16: Capture, escape, and timeout flowchart.

17 Scoring and Performance Evaluation System

The scoring system listens to simulation events and converts them into raw scores and performance percentages. The Police/Hunter is rewarded for capture, pressure, patrol coverage, dog assist, CCTV assist, fire assist, and containment. Prisoners are rewarded for escape and progress but penalized for capture, detection, dog pressure, fire contact, wall hits, and timeout.

Table 12: Scoring events

Event	Prisoner Score Effect	Police Score Effect
Prisoner escapes	Large bonus.	Escape-allowed penalty.
Police captures prisoner	Capture penalty.	Capture bonus and repeat capture bonus.
CCTV detects prisoner	Detection penalty.	CCTV assist opportunity.
Dog detects/engages prisoner	Zone or engagement penalty.	Dog assist opportunity.
Fire contact/elimination	Fire penalty.	Fire assist opportunity.
Timeout	Timeout penalty.	Containment advantage.
Patrol coverage	Not applicable.	Patrol coverage bonus.
Partial escape	Mixed result.	Mixed containment and escape penalty.

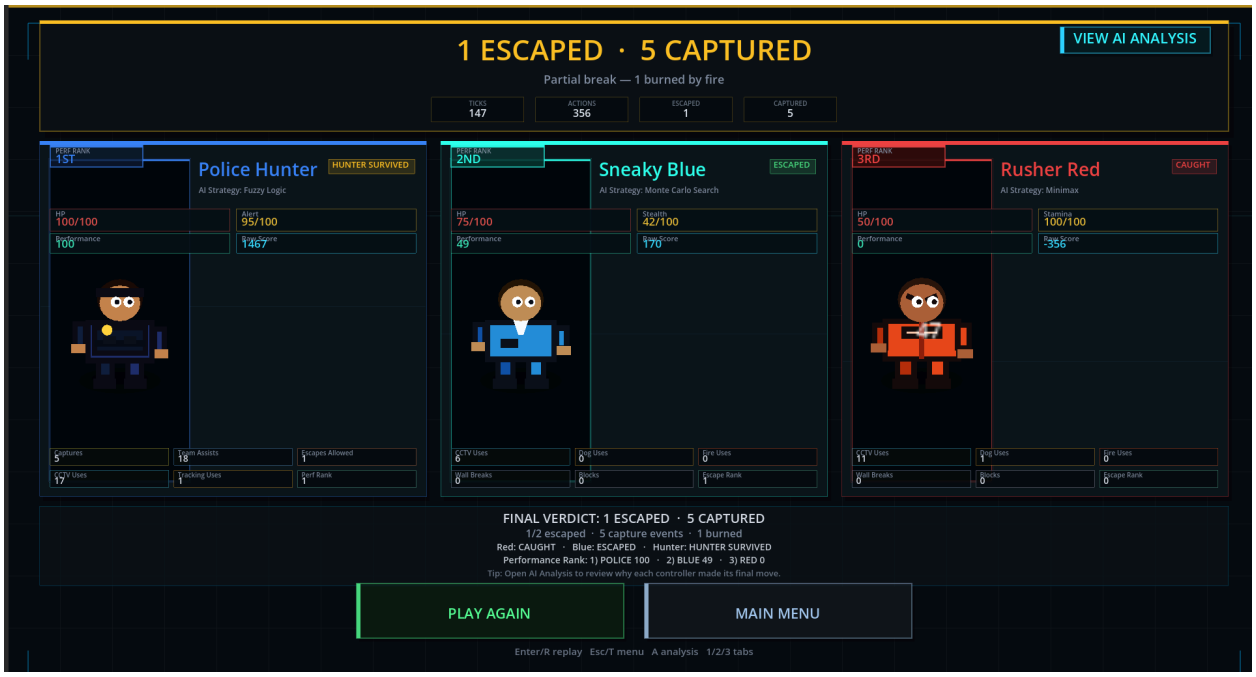


Fig 17: Final result screen with scores and ranking.

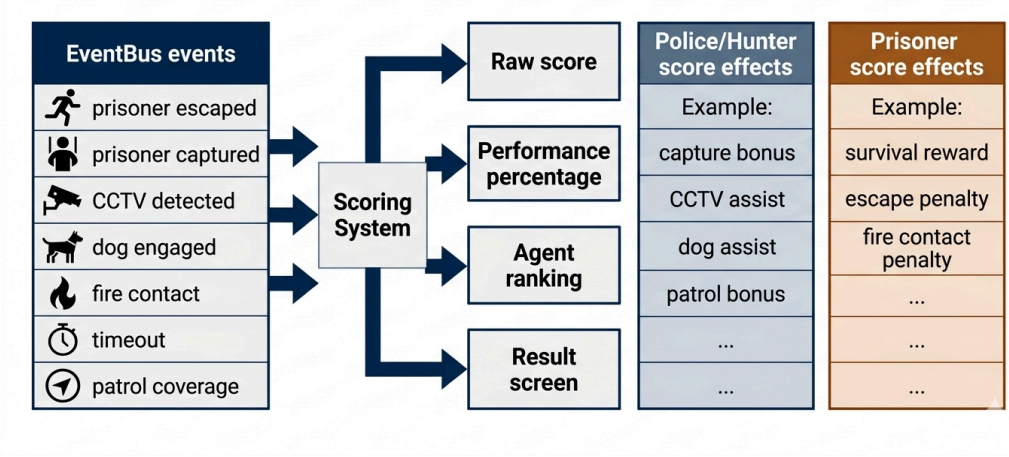


Fig 18:Scoring and Performance Evaluation System

18 AI Decision Visualization and Analysis Page

18.1 Live HUD

The HUD and overlays show timer, score events, agent status, fuzzy behaviour, candidate paths, danger, and vision information during the mat

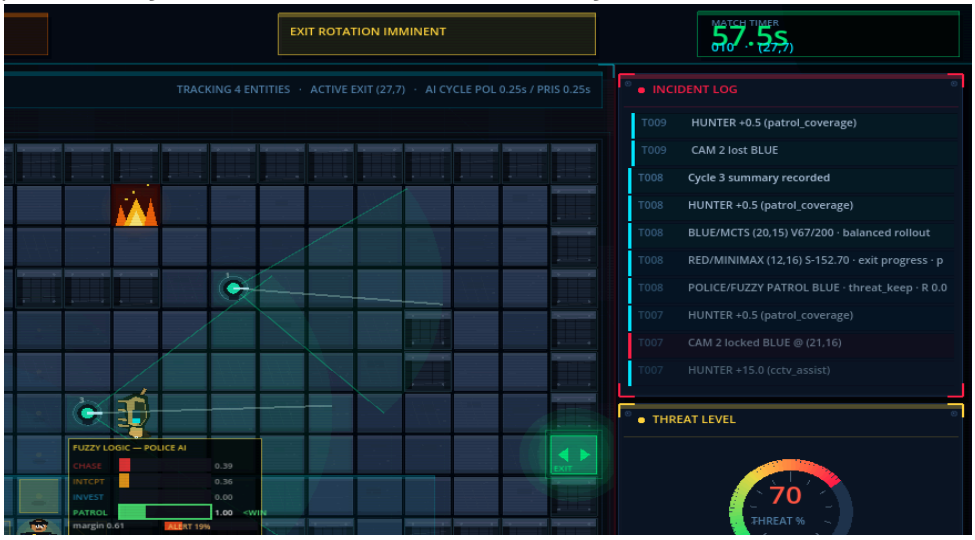
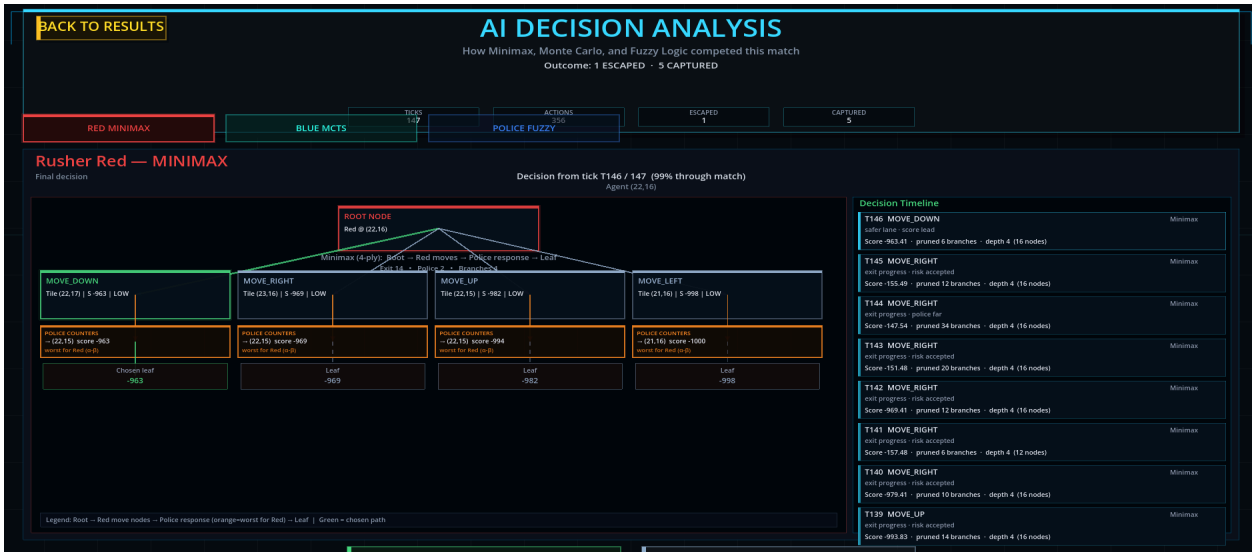
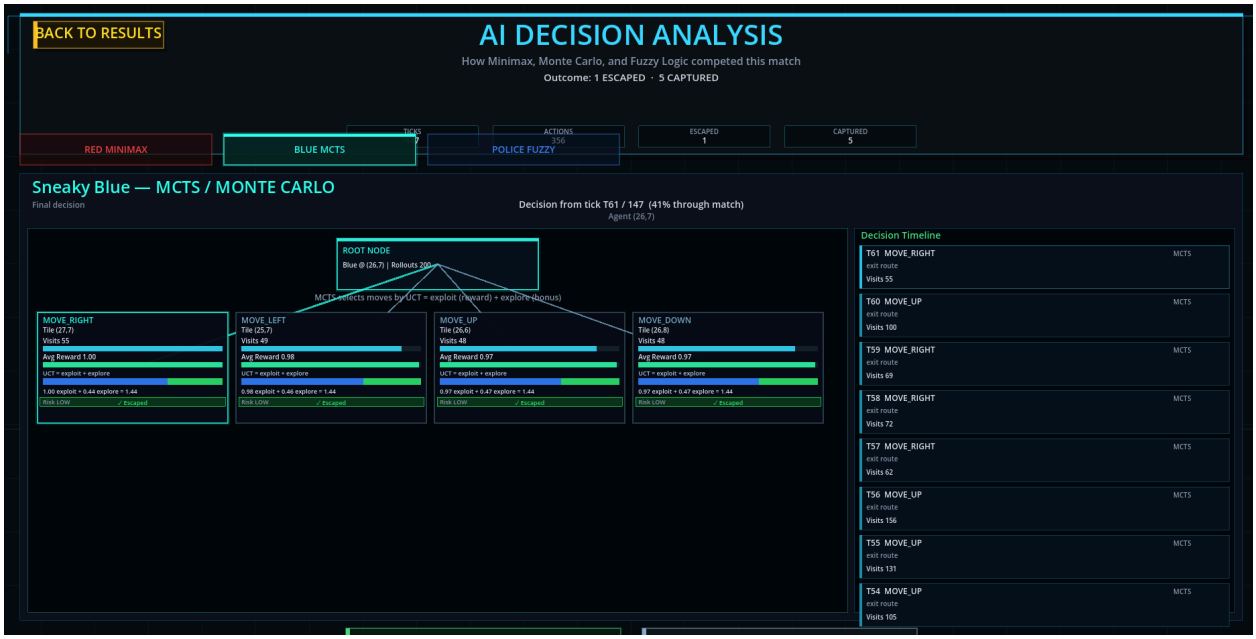


Fig 19: HUD with timer and AI status.

18.2 Decision Overlays

The result screen then summarizes ranking, captures, escapes, and AI decisions.


BACK TO RESULTS
AI DECISION ANALYSIS
How Minimax, Monte Carlo, and Fuzzy Logic competed this match
Outcome: 1 ESCAPED · 5 CAPTURED
TICKS 147
ACTIONS 356
ESCAPED 1
CAPTURED 5
RED MINIMAX
BLUE MCTS
POLICE FUZZY
Rusher Red — MINIMAX
Final decision
Decision from tick T146 / 147 (99% through match)
Agent (22,16)
ROOT NODE
Red @ (22,16)
Minimax (4-ply): Root → Red moves → Police response → Leaf
Top 14 → Police 2 → Branches 24
MOVE_DOWN
Tile (22,17) | S -963 | LOW
MOVE_RIGHT
Tile (23,16) | S -969 | LOW
MOVE_UP
Tile (22,15) | S -982 | LOW
MOVE_LEFT
Tile (21,16) | S -998 | LOW
POLICE COUNTERS
→ (22,15) score -963
worst for Red (e-jl)
POLICE COUNTERS
→ (22,15) score -969
worst for Red (e-jl)
POLICE COUNTERS
→ (22,15) score -994
worst for Red (e-jl)
POLICE COUNTERS
→ (21,16) score -1000
worst for Red (e-jl)
Chosen leaf
-963
Leaf
-969
Leaf
-982
Leaf
-998
Legend: Root → Red move nodes → Police response (orange=worst for Red) → Leaf | Green = chosen path
Decision Timeline
T146 MOVE_DOWN
Minimax
safer lane · score lead
Score -963.41 · pruned 6 branches · depth 4 (16 nodes)
T145 MOVE_RIGHT
Minimax
exit progress · risk accepted
Score -155.49 · pruned 12 branches · depth 4 (16 nodes)
T144 MOVE_RIGHT
Minimax
exit progress · police far
Score -147.54 · pruned 34 branches · depth 4 (16 nodes)
T143 MOVE_RIGHT
Minimax
exit progress · risk accepted
Score -151.48 · pruned 20 branches · depth 4 (16 nodes)
T142 MOVE_RIGHT
Minimax
exit progress · risk accepted
Score -969.41 · pruned 12 branches · depth 4 (16 nodes)
T141 MOVE_RIGHT
Minimax
exit progress · risk accepted
Score -157.48 · pruned 6 branches · depth 4 (12 nodes)
T140 MOVE_RIGHT
Minimax
exit progress · risk accepted
Score -979.41 · pruned 10 branches · depth 4 (16 nodes)
T139 MOVE_UP
Minimax
exit progress · risk accepted
Score -993.83 · pruned 14 branches · depth 4 (16 nodes)


BACK TO RESULTS
AI DECISION ANALYSIS
How Minimax, Monte Carlo, and Fuzzy Logic competed this match
Outcome: 1 ESCAPED · 5 CAPTURED
TICKS 147
ACTIONS 356
ESCAPED 1
CAPTURED 5
RED MINIMAX
BLUE MCTS
POLICE FUZZY
Sneaky Blue — MCTS / MONTE CARLO
Final decision
Decision from tick T61 / 147 (41% through match)
Agent (26,7)
ROOT NODE
Blue @ (26,7) | Rollouts 200
MCTS selects moves by UCT = exploit (reward) + explore (bonus)
MOVE_RIGHT
Tile (27,7)
Visits 55
Avg Reward 1.00
UCT = exploit + explore
1.00 exploit + 0.44 explore = 1.44
Risk LOW
✓ Escaped
MOVE_LEFT
Tile (25,7)
Visits 49
Avg Reward 0.98
UCT = exploit + explore
0.98 exploit + 0.46 explore = 1.44
Risk LOW
✓ Escaped
MOVE_UP
Tile (26,6)
Visits 48
Avg Reward 0.97
UCT = exploit + explore
0.97 exploit + 0.47 explore = 1.44
Risk LOW
✓ Escaped
MOVE_DOWN
Tile (26,8)
Visits 48
Avg Reward 0.97
UCT = exploit + explore
0.97 exploit + 0.47 explore = 1.44
Risk LOW
✓ Escaped
Decision Timeline
T61 MOVE_RIGHT
MCTS
exit route
Visits 55
T60 MOVE_UP
MCTS
exit route
Visits 100
T59 MOVE_RIGHT
MCTS
exit route
Visits 69
T58 MOVE_RIGHT
MCTS
exit route
Visits 72
T57 MOVE_RIGHT
MCTS
exit route
Visits 62
T56 MOVE_UP
MCTS
exit route
Visits 156
T55 MOVE_UP
MCTS
exit route
Visits 131
T54 MOVE_UP
MCTS
exit route
Visits 105

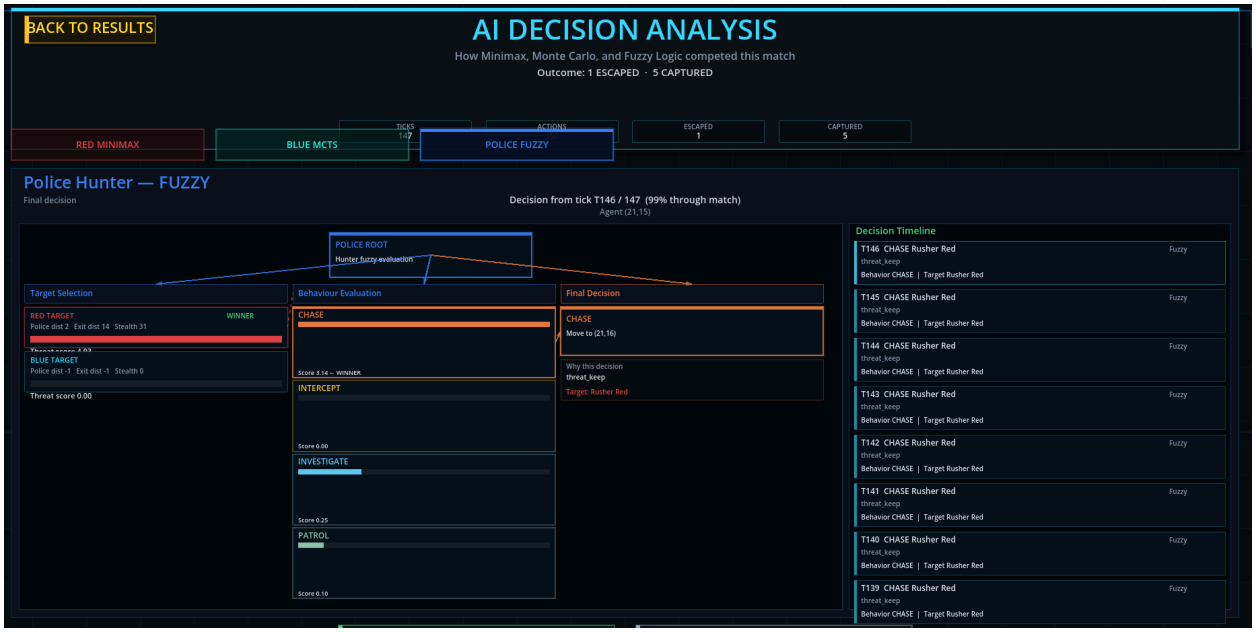


Fig 20: Decision overlay during gameplay.

18.3 AI Analysis Page

The AI analysis page is the most important educational screen. It presents Police/Hunter fuzzy target priority and behaviour scores, Red Minimax candidate moves and police response branches, and Blue MCTS visits, rewards, and UCT values.

19 Discussion

The Police/Hunter fuzzy logic system gives the defensive agent human-like behaviour because it reacts to several imperfect signals at once. Red Minimax provides deterministic adversarial planning, while Blue MCTS provides simulation-based exploration. CCTV, dog pressure, fire, walls, barriers, and exit rotation make the simulation more than a shortest-path problem.

The result screen and AI analysis page are essential because they explain why each agent acted as it did. This makes the project appropriate for an AI laboratory demonstration instead of only a visual game demo.

20 Conclusion

The Prison Break project demonstrates a complete multi-agent AI simulation in which a Police/Hunter fuzzy agent, Red Minimax prisoner, and Blue MCTS prisoner compete inside the same prison environment. The supporting systems—dog, CCTV, walls, doors, barriers, fire, exits, scoring, replay, benchmark, HUD, and AI analysis—make the environment dynamic and explainable.

The strongest contribution of the project is visible AI reasoning. With the required screenshots and generated diagrams inserted, the report can clearly show both implementation and algorithmic decision-making.

21 References

[1] Russell, S., and Norvig, P. *Artificial Intelligence: A Modern Approach*. Pearson.

[2] von Neumann, J., and Morgenstern, O. *Theory of Games and Economic Behavior*. Princeton University Press.

[3] Knuth, D. E., and Moore, R. W. "An Analysis of Alpha-Beta Pruning." *Artificial Intelligence*, 1975.

[4] Browne, C. B. et al. "A Survey of Monte Carlo Tree Search Methods." *IEEE Transactions on Computational Intelligence and AI in Games*, 2012.

[5] Zadeh, L. A. "Fuzzy Sets." *Information and Control*, 1965.

[6] Godot Engine Documentation. https://docs.godotengine.org/

[7] Hart, P. E., Nilsson, N. J., and Raphael, B. "A Formal Basis for the Heuristic Determination of Minimum Cost Paths." *IEEE Transactions on Systems Science and Cybernetics*, 1968.